RIS COURSE PROJECT · PAPER PRESENTATION

QPASE

Quantum-resistant password-authenticated searchable encryption for cloud storage

Jiang & Wang · IEEE Transactions on Information Forensics and Security, 2024

PART 1

The problem and the scheme

Shambhavi Gantla · 2026801014

PART 2

Building blocks and cost

Shambo Roy · 2026801007

PART 3

What we want to add

Iman Kalyan Chakraborty · 2026801003

The cloud search dilemma

Trust the server

The cloud decrypts and searches for you.

Problem: server reads your data

Download everything

Decrypt on your device, then search locally.

Problem: heavy traffic per query

Searchable encryption

The server searches ciphertext without reading it.

Problem: strong key on every device

A long random key cannot be memorised. It must be stored and copied to each device. **Can a password alone do the job?**

PART 1 · THE GAP

From keys to passwords to post-quantum

SSE · since 2000

Key-based search

User holds a high-entropy key.

Key must live on each device

PASE · Chen et al. 2021

Password-based search

User holds only a password. Servers help rebuild the key.

Discrete-log: broken by Shor

QPASE · this paper, 2024

Quantum-safe password search

Same password-only promise, rebuilt on lattices.

No known quantum attack on LWE

The authors claim the **first** password-authenticated searchable encryption that resists a quantum adversary.

QPASE at a glance



A quantum attacker may try: offline password guessing · online password guessing · chosen-keyword attacks

Four phases, one password

01

Register

Derive user key Ku. Give servers a public key.

02

Login

Rebuild Ku from the password. A typo is caught at once.

03

Outsource

Encrypt file and keyword index locally. Sign. Upload.

04

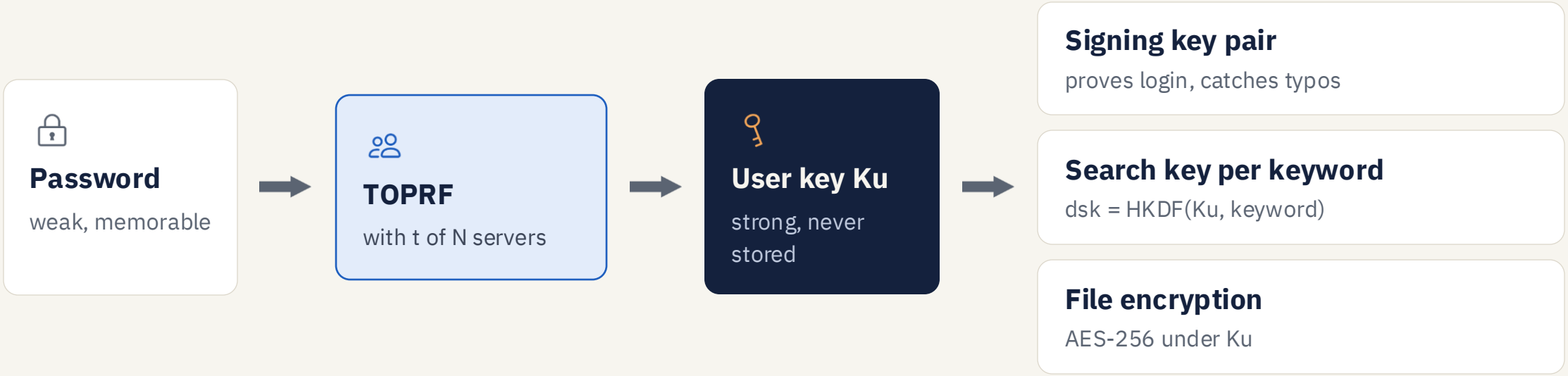
Retrieve

Send a keyword search key. Get matching ciphertexts back.

Server key update: shares refresh every epoch, the user does nothing.

Multi-keyword: AND, OR and subset queries are supported.

How one password becomes every key



Ku needs **both** the password and t servers. Either one alone is useless to an attacker.

The building blocks

Building block	Where it is used	Why it is needed
Decision-LWE	Under TOPRF, PRF and signatures	Believed hard even for quantum computers
Lattice TOPRF	Register and Login	Strong key from a password no server sees
Robust extractor	On the TOPRF output	Same password always gives the same key
HKDF and PRF	Outsource and Retrieve	A separate search key for every keyword
Dilithium signature	Every message after login	Proves identity, blocks tampering
Key generation (DKG)	Setup and server key update	No trusted dealer; shares can be refreshed
AES-256	File and index encryption	Still 128-bit secure against Grover

The three highlighted rows carry the scheme. The next three slides take them one by one.

Decision-LWE: the hardness foundation

World 1: secret s hidden under small noise e

(A , A·s + e)

cannot be told apart

World 2: pure randomness

(A , random b)

WHAT

Noisy linear equations look random. Stripping the noise is the hard problem.

WHY

Shor breaks factoring and discrete-log. No efficient quantum attack on LWE is known.

WHERE IN QPASE

The TOPRF, the PRF and the Dilithium signature all rest on it.

Chosen parameters: dimension n = 595, modulus q ≈ 2^28, giving 128-bit quantum security (n = 512 gives 92-bit).

TOPRF: from password to strong key

01 · USER

Blind

Hide the password under fresh randomness.

02 · t SERVERS

Evaluate

Each applies its key share, without seeing the input.

03 · USER

Combine

Merge t replies and remove the blinding.

OUTPUT

User key Ku

High entropy, same every login.

Oblivious

Servers never see the password or the key.

Threshold

Fewer than t corrupt servers learn nothing.

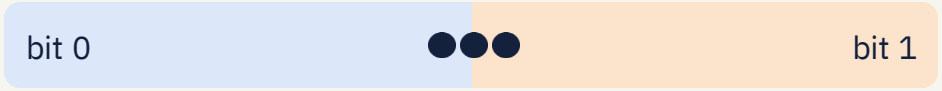
No offline guessing

Every guess needs the servers, and they count attempts.

Robust extractor: removing lattice noise

The problem. Lattice outputs are noisy. Three logins with the same password land on three nearby values.

Plain rounding: values straddle a boundary, keys disagree



With a 1-bit hint: boundary shifts, all three agree

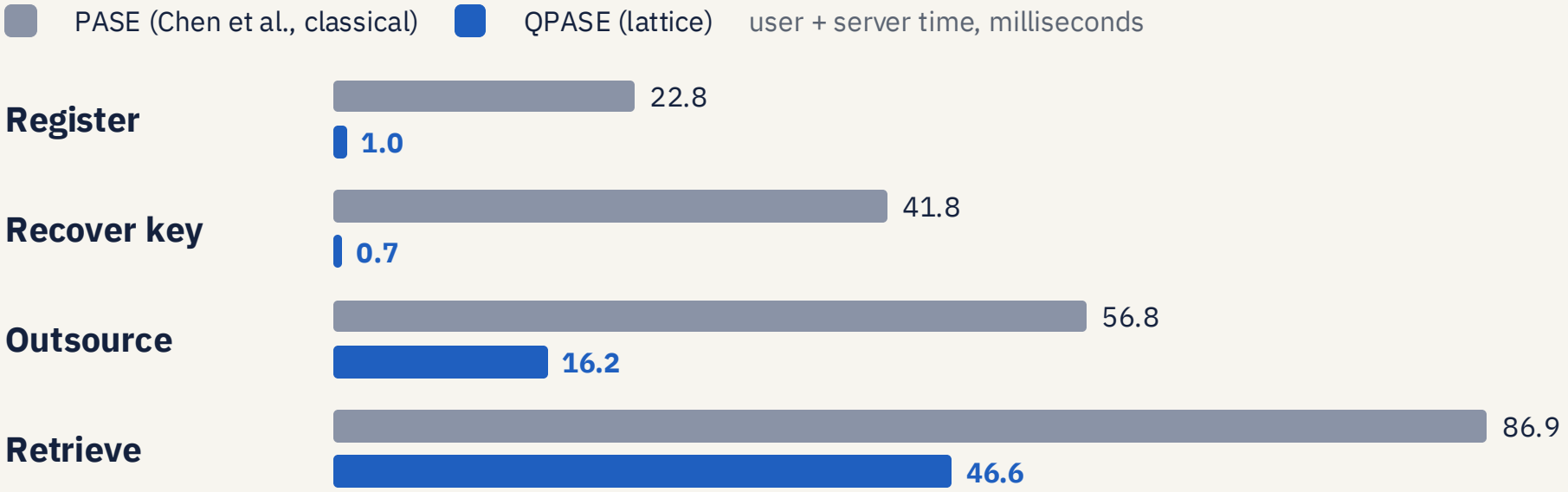


Why QPASE needs it

- Ku must be identical at every login.
- Otherwise the public-key check fails and files cannot be decrypted.
- The hint reveals a coarse range only, not the key.

This is the paper's own change to the lattice TOPRF: the "variant-TOPRF".

Computation cost per phase



No exponentiations. Lattice work is matrix multiply and add: key recovery is about 60× cheaper, search phases about 2× to 3.5×.

Source: paper Table IV · 100 KB file, N = t = 2, 128-bit security

Communication overhead

REGISTER · ONCE

3,400×

more data than PASE

160 bytes → about 550 KB

RECOVER KEY · EVERY LOGIN

1,170×

more data than PASE

224 bytes → about 262 KB

OUTSOURCE / RETRIEVE

+8–16%

for a 100 KB file

and one round instead of two

Why: lattice elements are large, and they go to every server, so traffic grows linearly with N.

Authors' own caveat: not suited to low-bandwidth settings. The login cost is fixed, so it matters less for big files.

Source: paper Table IV, converted from bits · N = t = 2

QPASE against related schemes

Scheme	Threshold	Quantum-safe	Search	Passwords	Exp. (S / U)
Jarecki et al. 2014 (PASS)	(t, N)	No	No	Uniform	4 / 11
Das et al. 2020 (DPaSE)	(N, N)	No	No	Uniform	4 / 10
Chen et al. 2021 (PASE)	(2, 2)	No	Yes	Uniform	8 / 3
Roy et al. 2021	(t, N)	Yes	No	Uniform	0 / 0
Jiang et al. 2023 (QPause)	(t, N)	Yes	No	Zipf	0 / 0
QPASE (this paper)	(t, N)	Yes	Yes	Zipf	0 / 0

Only QPASE is quantum-safe **and** searchable, with any threshold and a realistic (Zipf) password model.

Source: paper Table V (selected rows) · Exp. = exponentiations, server / user

PART 3 · OUR CONTRIBUTION

What QPASE leaves open in search

PROBLEM 1

The search key is permanent

To search a keyword, the user hands the server dsk, the key for that keyword. The server can keep it and test every later upload against it, without the user searching again.

No forward privacy

PROBLEM 2

Every search is a full scan

Each file carries its own locked tag. The server has no way to know which tags match, so it tries dsk on every stored tag, one by one.

Cost grows with total storage, not with matches

Both problems come from the same place: the tags only reveal a match when a key is tried on them. **We replace the tags with a lookup table.**

Our reading of the Outsource and Retrieve steps in Section IV of the paper.

PART 3 · OUR CONTRIBUTION

Our proposal: a counter-based search index

QPASE TODAY

User → **permanent key dsk** → **Server**

- Server tries dsk on every stored tag.
- Server keeps dsk and can match future uploads.

OUR PROPOSAL

User → **one-time labels 1 to c** → **Server**

- Server looks each label up in a table. No scan.
- The next upload uses a label it cannot compute.

Server search work	Forward privacy	Quantum safety
all entries → matches only	**no → yes**	**kept: hash and PRF only**

Each keyword gets a counter. The i-th file with that keyword is stored under **label i = PRF(K_w, i)**. The key K_w = HKDF(Ku, keyword) is the old dsk, but it now **stays on the device**.

Expected outcomes, to be confirmed by our implementation.

From an inverted index to an encrypted dictionary

A PLAIN INVERTED INDEX

Keyword	Files
invoices	A, C, D
contracts	B, E

It maps each keyword to its files. Search engines use this shape, because a search starts from a keyword.

Three changes give our index

1. **One row per keyword and file pair**
   Each list is split up. "invoices" becomes three separate rows.
2. **A label replaces the keyword**
   Row i of a keyword gets label $i = PRF(K_w, i)$.
3. **The file pointer is encrypted**
   Only the user can read where a row points.

Why flatten it

One row per keyword would show the server how many keywords exist and how many files each has, before any search.

How the server stores it

As an ordinary hash table, looked up by exact label. The literature calls this an encrypted dictionary.

It still answers keyword to files, but the server no longer sees which rows form a list.

What the index looks like

WHAT ONLY THE USER CAN WORK OUT

Keyword	i	Label = PRF(K_w, i)	File
invoices	1	`3b90…5e`	A
invoices	2	`a47d…09`	C
invoices	3	`d6f1…72`	D
contracts	1	`1f7c…a3`	B
contracts	2	`8e12…c4`	E

WHAT THE SERVER STORES

Label	Entry
`1f7c…a3`	encrypted pointer to file B
`3b90…5e`	encrypted pointer to file A
`8e12…c4`	encrypted pointer to file E
`a47d…09`	encrypted pointer to file C
`d6f1…72`	encrypted pointer to file D

Server view

Five unrelated rows. It cannot tell which share a keyword, or how many keywords there are.

Counter table

invoices = 3, contracts = 2. Encrypted under a key from Ku, kept on the servers, fetched at login.

Labels are shortened and made up for illustration. Real labels are 256-bit PRF outputs.

The same search in both schemes

QPASE TODAY

1. **User sends one key**
   dsk = HKDF(Ku, "invoices"), with a signature.
2. **Server tries dsk on all five tags**
   A ✓ B ✗ C ✓ D ✓ E ✗
   Each try is a decryption, a signature check and a PRF check.
3. **Server returns A, C, D**
   It keeps dsk and can test every later upload.

OUR PROPOSAL

1. **User sends three labels**
   `3b90…5e`, `a47d…09`, `d6f1…72`, with a signature.
2. **Server looks up three rows**
   A ✓ C ✓ D ✓
   Rows for B and E are never touched.
3. **Server returns A, C, D**
   It keeps three used labels. They match nothing new.

Server work here
5 tries → 3 lookups

With 1,000,000 stored files, 3 matches
1,000,000 tries → 3 lookups

What the server keeps
a reusable key → used labels

Search for "invoices" with files A to E stored. The result is the same in both schemes.

A small example: search, then upload

1 Search "invoices"

- Counter says c = 3.
- User computes labels 1 to 3 and sends
  `3b90…5e, a47d…09, d6f1…72`
- Server does 3 lookups and returns files A, C and D, still encrypted.

2 Upload file F, "invoices"

- Counter goes from 3 to 4.
- User computes label 4 = `62ab…1d`
- Server adds one row
  `62ab…1d` → pointer to F
- User saves the new counter table.

3 What the server learned

- It knows rows `3b90`, `a47d` and `d6f1` belong together.
- Until the user searches "invoices" again, it cannot link the new row `62ab` to them. It never had K_w, so it cannot compute label 4.
- In QPASE today it would test F with the kept dsk and see the match at once.

Continues the same example. File contents stay encrypted under Ku throughout.

What it costs and how we will test it

Costs we accept

- Counter table grows with distinct keywords
- One extra small download per login
- Search message grows with the number of matches
- Two devices at once must sync the counter
- The server still sees which files a search returns

Project plan

1. **Reproduce**
   Implement QPASE with the paper's parameters.
2. **Improve**
   Replace the per-file tags with the counter index.
3. **Measure**
   Search time against storage size, for N = 3, 5, 7 servers.

WHAT WE WILL DELIVER

A working QPASE. Our implementation of the paper, with its parameters.

QPASE with the counter index. The same code, with the new Outsource and Retrieve.

A comparison. Search time against storage size for both, and a test that old labels miss new uploads.

SUMMARY

QPASE makes password-only encrypted search quantum-safe.

We aim to make its search forward-private and faster, with a counter-based index.

THE PAPER

Lattice TOPRF + robust extractor turn a password into a strong key.

THE COST

Faster to compute than PASE, far heavier on the network.

OUR ADDITION

A counter-based search index: table lookups in place of a scan, and one-time labels.

Questions?